APACHE KAFKA TRAINING PROGRAM

DAY 3 — HANDS-ON LAB GUIDE

Monitoring, Troubleshooting & Diagnostics

7 Hands-On Labs | Structured Exercises | Challenge Scenarios

Prerequisites: Kafka running on localhost:9092 | Day 1 & 2 complete

Lab Overview & Prerequisites

Lab #	Title	Duration	Level	Module
Lab 1	Kafka Metrics Exploration with JMX & CLI	30 min	Beginner	Module 1
Lab 2	Setting Up Consumer Lag Monitoring	30 min	Intermediate	Module 1-2
Lab 3	Simulating and Resolving Consumer Lag	45 min	Intermediate	Module 3
Lab 4	Inducing and Diagnosing Under-Replicated Partitions	45 min	Advanced	Module 3
Lab 5	Kafka Broker Log Analysis	30 min	Intermediate	Module 4
Lab 6	Producer Troubleshooting Scenarios	30 min	Intermediate	Module 3
Lab 7	End-to-End Monitoring Dashboard Challenge	30 min	Advanced	Module 2

Environment Prerequisites

- Kafka 3.x installed and running on localhost:9092
- Java 11+ installed (for jconsole access)
- kafka-topics.sh, kafka-console-producer.sh, kafka-consumer-groups.sh available in PATH
- Python 3.x with confluent-kafka library installed: pip install confluent-kafka
- curl installed for HTTP API calls
- Prometheus + JMX Exporter (optional, required for Lab 7)

Before You Begin — Verify Environment

```
# Confirm Kafka is running
kafka-topics.sh --bootstrap-server localhost:9092 --list
```

```
# Create a test topic for labs (replication factor 3 requires 3-broker cluster)
kafka-topics.sh --bootstrap-server localhost:9092 --create \
  --topic kafka-lab-topic --partitions 6 --replication-factor 1
```

```
# NOTE: Use --replication-factor 3 if you have a 3-broker cluster
```

LAB 1: Kafka Metrics Exploration with JMX & CLI | Duration: 30 minutes | Level: Beginner

Objective: Learn to read and interpret critical Kafka metrics using CLI tools and JMX MBeans. By the end of this lab, you should be able to identify the health status of a Kafka broker from its metrics.

Part A: CLI-Based Metrics

Step 1: List all topics and check their state

```
kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Step 2: Describe a topic and examine partition distribution

```
kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic kafka-lab-topic
```

Look at the output. Identify:

- Leader — which broker ID is the leader for each partition
- Replicas — which broker IDs hold replicas
- Isr — which replicas are in-sync (should match Replicas for healthy topic)

Expected Output: Isr should list the same brokers as Replicas for a healthy topic. Note any discrepancy.

Step 3: Check for under-replicated partitions

```
kafka-topics.sh --bootstrap-server localhost:9092 --describe --under-replicated-
partitions
```

Expected Output: No output expected on a healthy cluster. If you see partitions listed, investigate.

Step 4: Check consumer group status

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --all-groups
```

Part B: JMX Metrics via jconsole

Step 5: Open jconsole and connect to the Kafka broker

```
# Start JMX if not already enabled
export JMX_PORT=9999
# Restart Kafka broker, then:
jconsole localhost:9999
```

Step 6: Navigate to the following MBeans and record the values:

MBean Path	Metric to Record	Your Value	Healthy?
kafka.server > ReplicaManager > UnderReplicatedPartitions	Value	_____	_____
kafka.controller > KafkaController > ActiveControllerCount	Value	_____	_____
kafka.controller > KafkaController > OfflinePartitionsCount	Value	_____	_____
kafka.server > ReplicaManager > LeaderCount	Value	_____	_____
kafka.network > RequestMetrics > TotalTimeMs (Produce)	99thPercentile	_____	_____

Part C: Reflection Questions

1. What value should ActiveControllerCount show for a single-broker cluster? What would be wrong if it showed 0?

Answer: _____

2. What is the difference between Replicas and Isr in a topic description?

Answer: _____

3. If UnderReplicatedPartitions shows a value of 3, what does that mean and what should you do first?

Answer: _____

Lab 1 Completion Checklist

- ☐ Listed all topics and described kafka-lab-topic
- ☐ Identified leader, replicas, and ISR for each partition
- ☐ Confirmed no under-replicated partitions
- ☐ Opened jconsole and recorded all 5 MBean values in the table
- ☐ Answered all 3 reflection questions

LAB 2: Setting Up Consumer Lag Monitoring | Duration: 30 minutes | Level: Intermediate

Objective: Configure consumer lag monitoring using kafka-consumer-groups.sh and set up a continuous monitoring loop. Learn to interpret lag metrics in real-time.

Part A: Create Test Data and Consumer Group

Step 1: Produce 10,000 messages to the test topic

```
kafka-producer-perf-test.sh \
  --topic kafka-lab-topic \
  --num-records 10000 \
  --record-size 256 \
  --throughput 1000 \
  --producer-props bootstrap.servers=localhost:9092
```

Expected Output: Output shows: 10000 records sent, throughput ~1000 records/sec, avg latency

Step 2: Start a consumer group (run in a new terminal window)

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic kafka-lab-topic \
  --group lab2-consumer-group \
  --from-beginning
```

Step 3: While consumer is running, check the lag in another terminal

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe --group lab2-consumer-group
```

Record the following from the output:

Field	Value You Observed
Total number of partitions listed	
Maximum LAG across all partitions	
Any partition with CONSUMER-ID = '-'?	
HOST column value	

Part B: Continuous Lag Monitoring

Step 4: Set up a continuous monitoring watch (in a separate terminal)

```
watch -n 3 'kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group lab2-consumer-group'
```

Step 5: Produce another burst of 50,000 messages while watching

```
kafka-producer-perf-test.sh \
  --topic kafka-lab-topic \
  --num-records 50000 \
  --record-size 256 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:9092
```

Expected Output: You should see the LAG values briefly increase as messages flood in, then decrease as the consumer catches up. If the consumer is fast enough, lag may stay near zero.

Part C: Reset Offsets Exercise

Step 6: Stop the consumer (Ctrl+C). Now reset the consumer group offset to the beginning

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group lab2-consumer-group \
  --topic kafka-lab-topic \
  --reset-offsets --to-earliest --execute
```

Step 7: Check the lag now (with no consumer running)

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group lab2-consumer-group
```

Expected Output: LAG should now show the full message count for each partition (consumer is behind by all messages). CONSUMER-ID should show '-' since no consumer is active.

Tip: When CONSUMER-ID shows '-', it means no consumer instance is currently assigned to that partition. The offsets are still stored, but no one is consuming.

Lab 2 Completion Checklist

- ☐ Produced 10,000 test messages successfully
- ☐ Started consumer group and observed it consuming
- ☐ Described consumer group and filled in the observation table
- ☐ Ran continuous watch and observed lag during a message burst
- ☐ Reset offsets to earliest and confirmed lag reflects all messages

LAB 3: Simulating and Resolving Consumer Lag | Duration: 45 minutes | Level: Intermediate

Objective: Simulate a real-world consumer lag scenario using a slow Python consumer. Observe lag growing in real-time, then apply remediation strategies to resolve it.

Warning: This lab requires Python 3 with confluent-kafka installed. Run: pip3 install confluent-kafka

Part A: Create a Slow Consumer

Step 1: Create the slow consumer Python script

```
# Save this as slow_consumer.py
from confluent_kafka import Consumer, KafkaException
import time, sys

conf = {
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'slow-consumer-group',
    'auto.offset.reset': 'earliest',
    'enable.auto.commit': True,
    'auto.commit.interval.ms': 5000
}

c = Consumer(conf)
c.subscribe(['kafka-lab-topic'])
count = 0
try:
    while True:
        msg = c.poll(timeout=1.0)
        if msg is None: continue
        if msg.error(): print(f'Error: {msg.error()}'); continue
        count += 1
        time.sleep(0.05)  # 50ms delay = max 20 msgs/sec
        if count % 100 == 0:
            print(f'Processed {count} messages, latest offset: {msg.offset()}')
except KeyboardInterrupt:
    c.close()
```

Step 2: Reset the consumer group offset to the beginning first

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group slow-consumer-group \
  --topic kafka-lab-topic \
  --reset-offsets --to-earliest --execute
```

Part B: Induce Lag by Producing Faster Than Consuming

Step 3: Start the slow consumer (Terminal 1)

```
python3 slow_consumer.py
```

Step 4: While the slow consumer runs, flood the topic (Terminal 2)

```
kafka-producer-perf-test.sh \
  --topic kafka-lab-topic \
  --num-records 100000 \
  --record-size 256 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:9092
```

Step 5: Monitor lag growing (Terminal 3)

```
watch -n 2 'kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group slow-consumer-group | grep -E "TOPIC|kafka-lab"'
```

Record your observations every 30 seconds for 2 minutes:

Time (seconds)	Total Lag (approx)	Max Partition Lag	Observation
0 seconds			
30 seconds			
60 seconds			
90 seconds			
120 seconds			

Part C: Remediation Strategies

Step 6: Remediation Option A — Skip ahead (offset reset to latest)

Warning: This causes message loss — all messages produced while the consumer was slow will be skipped. Only use in non-critical scenarios.

```
# Stop the slow consumer first, then:
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group slow-consumer-group --topic kafka-lab-topic \
  --reset-offsets --to-latest --execute
```

Expected Output: Lag should drop to 0 immediately. Restart the slow consumer and verify lag stays near 0.

Step 7: Remediation Option B — Speed up the consumer

Modify slow_consumer.py — change the sleep from 0.05 to 0.001 (1ms) and restart:

```
# Edit slow_consumer.py: time.sleep(0.001)  # 1ms = up to 1000 msgs/sec
python3 slow_consumer.py
```

Expected Output: With 1ms processing time, the consumer can handle up to ~1000 messages/sec, which should outpace the typical producer rate. Observe lag decreasing.

Lab 3 Reflection Questions

4. What is the maximum sustainable message rate for a consumer with 50ms processing time per message with 6 partitions?

Answer: _____

5. What are the trade-offs between resetting to latest vs. speeding up the consumer?

Answer: _____

6. How many consumer instances would you need to handle 10,000 messages/sec with 50ms processing time per message?

Answer: _____

Lab 3 Completion Checklist

- ☐ Created slow_consumer.py with 50ms artificial delay
- ☐ Observed lag growing in the monitoring watch window
- ☐ Filled in the lag observation table with real values
- ☐ Applied offset reset remediation and confirmed lag dropped to 0
- ☐ Modified consumer speed and verified faster consumption rate

LAB 4: Inducing and Diagnosing Under-Replicated Partitions | Duration: 45 minutes | Level: **Advanced**

Objective: Simulate an Under-Replicated Partition (URP) condition by manipulating broker behavior. Learn to identify URPs, understand their impact, and follow the diagnostic workflow to resolution.

Warning: This lab requires a 3-broker Kafka cluster. If you only have a single broker, follow the simulation steps theoretically and note what you would observe.

Part A: Setup Multi-Topic with Replication (3-Broker Cluster)

Step 1: Create a topic with replication factor 3

```
kafka-topics.sh --bootstrap-server localhost:9092 --create \
  --topic replicated-topic \
  --partitions 3 \
  --replication-factor 3
```

Step 2: Verify all replicas are in-sync

```
kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic replicated-topic
```

Expected Output: All partitions should show Replicas: 0,1,2 and Isr: 0,1,2 (or similar broker IDs). No under-replicated partitions.

Part B: Simulate a Broker Failure

Step 3: Stop one of the follower brokers (e.g., broker on port 9094)

```
# Find and stop a specific broker
# For a local multi-broker setup:
kafka-server-stop.sh  # OR kill the process
# kill $(lsof -t -i:9094)  # Kill broker on port 9094
```

Step 4: Immediately check for URPs

```
kafka-topics.sh --bootstrap-server localhost:9092 --describe \
  --under-replicated-partitions
```

Expected Output: You should now see partitions with fewer replicas in Isr than in Replicas. Example: Replicas: 0,1,2 Isr: 0,1

Step 5: Record the URP state

Metric	Before Broker Stop	After Broker Stop
Under-Replicated Partitions count	0	

Metric	Before Broker Stop	After Broker Stop
ISR for replicated-topic partition 0		
ISR for replicated-topic partition 1		
ISR for replicated-topic partition 2		

Part C: Diagnose the URP

Step 6: Run the full diagnostic workflow

7. Check broker availability

```
kafka-broker-api-versions.sh --bootstrap-server localhost:9093 2>&1
# Try each broker port — the stopped one will fail
```

8. Check replica manager metrics on remaining brokers

```
# Via jconsole: kafka.server > ReplicaManager > UnderReplicatedPartitions
```

9. Check broker logs for replication errors

```
grep -i 'replica\|isr\|replication\|dead' /opt/kafka/logs/server.log | tail -50
```

Step 7: Note what log messages you see related to the stopped broker

Log observations:
_____
_____
_____
_____
_____

Part D: ISR Recovery

Step 8: Restart the stopped broker

```
kafka-server-start.sh /opt/kafka/config/server-2.properties &
```

Step 9: Monitor ISR recovery in real-time

```
watch -n 3 'kafka-topics.sh --bootstrap-server localhost:9092 \
  --describe --under-replicated-partitions'
```

Expected Output: Within 30-60 seconds, the restarted broker should rejoin the ISR. The under-replicated-partitions output should eventually show no output (empty = all healthy).

Step 10: Record recovery time

- Time broker was stopped: _____
- Time ISR fully recovered (no URPs): _____
- Total recovery duration: _____

> **Tip:** The recovery time depends on how much data the follower needs to catch up. The parameter replica.lag.time.max.ms (default 10s) controls when a follower is removed from ISR. A follower that is too slow to keep up will be removed from ISR even if it's still running.

Lab 4 Completion Checklist

- ☐ Created replicated-topic with RF=3 and verified all ISRs are healthy
- ☐ Stopped a broker and immediately observed URPs appearing
- ☐ Filled in the URP state observation table
- ☐ Ran the diagnostic workflow (broker check, metrics, log grep)
- ☐ Restarted the broker and confirmed ISR recovery
- ☐ Recorded the recovery time

LAB 5: Kafka Broker Log Analysis | Duration: 30 minutes | Level: Intermediate

Objective: Develop the ability to quickly extract meaningful diagnostic information from Kafka broker logs. Practice identifying error patterns, ISR changes, and timeline construction from logs.

Part A: Generate Log Events

Step 1: Produce and consume to generate activity in the logs

```
# Produce messages
kafka-producer-perf-test.sh --topic kafka-lab-topic --num-records 5000 \
  --record-size 512 --throughput 500 \
  --producer-props bootstrap.servers=localhost:9092
```

```
# Consume messages
kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic kafka-lab-topic --group loganalysis-group --from-beginning \
  --max-messages 5000
```

Part B: Log Extraction Exercises

Complete each grep exercise below and record your observations:

Exercise B-1: Find all ISR changes

```
grep -i 'shrinking isr\|expanding isr' /opt/kafka/logs/server.log | tail -20
```

Record your findings:

Findings: _____
_____
_____

Exercise B-2: Find any ERROR entries from the last hour

```
grep 'ERROR' /opt/kafka/logs/server.log | tail -30
```

List any errors found (or write 'No errors found'):

Errors: _____
_____
_____

Exercise B-3: Count warnings per minute to find traffic spikes

```
grep 'WARN' /opt/kafka/logs/server.log | awk '{print $1, $2}' | cut -c1-16 | sort |
uniq -c | sort -rn | head -10
```

Which minute had the most warnings? _____

Exercise B-4: Find consumer group coordinator activity

```
grep -i 'GroupCoordinator\|rebalance\|heartbeat' /opt/kafka/logs/server.log | tail -20
```

Part C: Log Pattern Recognition Exercise

Match the log message to the incident type. Draw a line or write the letter:

Log Message (Abbreviated)	Incident Type
[Partition orders-3] Shrinking ISR from 1,2,3 to 1	A. Consumer rebalancing
java.io.IOException: No space left on device	B. Broker failure detected
Member consumer-1 failed heartbeat check	C. URP / replica falling behind
[KafkaController]: Broker 2 has been marked dead	D. Disk full — critical
org.apache.kafka.common.errors.CorruptRecordException	E. Data corruption

Answers: ___1→___ , ___2→___ , ___3→___ , ___4→___ , ___5→___

> **Tip:** Correct answers: 1→C, 2→D, 3→A, 4→B, 5→E. Recognizing these patterns quickly is the hallmark of an experienced Kafka L2/L3 engineer.

Lab 5 Completion Checklist

- ☐ Generated log activity with producer and consumer
- ☐ Found and recorded ISR change events
- ☐ Identified any ERROR entries (or confirmed none present)
- ☐ Ran the per-minute warning count query
- ☐ Completed the log pattern recognition matching exercise

LAB 6: Producer Troubleshooting Scenarios | Duration: 30 minutes | Level: Intermediate

Objective: Simulate and diagnose common producer failures including buffer exhaustion, message size violations, and timeout conditions. Learn to read producer metrics and resolve configuration issues.

Scenario A: RecordTooLargeException

Step 1: Try to produce a message that exceeds the default max.request.size

```
# Generate a 2MB message (default max.request.size is 1MB)
python3 -c "
from confluent_kafka import Producer
p = Producer({'bootstrap.servers': 'localhost:9092'})
large_msg = 'X' * 2097152  # 2MB
try:
    p.produce('kafka-lab-topic', value=large_msg.encode())
    p.flush()
    print('Message sent successfully')
except Exception as e:
    print(f'Error: {e}')
"
```

Expected Output: You should receive a MessageSizeTooLarge or RecordTooLargeException error.

Step 2: Fix the configuration — increase max.request.size

```
python3 -c "
from confluent_kafka import Producer
p = Producer({
    'bootstrap.servers': 'localhost:9092',
    'message.max.bytes': 5242880  # 5MB
})
large_msg = 'X' * 2097152  # 2MB
p.produce('kafka-lab-topic', value=large_msg.encode())
p.flush()
print('Message sent with larger limit')
"
```

Warning: You may also need to set message.max.bytes on the BROKER (server.properties) for large messages to be accepted. Both producer and broker limits must be adjusted.

Scenario B: Producer Metrics Inspection

Step 3: Create a producer that logs its own metrics

```
# Save as producer_metrics.py
from confluent_kafka import Producer
import json

def print_stats(stats_json):
    stats = json.loads(stats_json)
    print(f"Txmsgs: {stats['txmsgs']}, Txbytes: {stats['txbytes']}")
    print(f"Msg queue depth: {stats['msg_cnt']}")

p = Producer({
    'bootstrap.servers': 'localhost:9092',
    'statistics.interval.ms': 2000,  # Stats every 2 seconds
    'stats_cb': print_stats
})

for i in range(10000):
    p.produce('kafka-lab-topic', value=f'message-{i}'.encode())
    p.poll(0)

p.flush()
print('Done')
```

```
python3 producer_metrics.py
```

Record the metrics output:

- Total messages sent (txmsgs): _____
- Total bytes sent (txbytes): _____
- Maximum queue depth observed (msg_cnt): _____

Scenario C: Troubleshooting Checklist

For each producer issue, identify the metric to check and the fix:

Issue	Metric to Check	Fix (config change)
Messages taking too long to send	request-latency-avg	
Producer buffer filling up	buffer-available-bytes	
High error rate	record-error-rate	
Low throughput	record-send-rate	

Lab 6 Completion Checklist

- ☐ Triggered RecordTooLargeException with a 2MB message
- ☐ Fixed by increasing message.max.bytes and verified message sent successfully

- ☐ Ran producer_metrics.py and recorded statistics output
- ☐ Completed the troubleshooting checklist table

LAB 7: End-to-End Monitoring Dashboard Challenge | Duration: 30 minutes | Level: Advanced

Objective: Synthesize everything from Day 3 into a complete incident response scenario. You will be given a simulated 'broken' cluster state and must diagnose all issues, prioritize them by severity, and document your remediation plan.

The Scenario

You are the on-call Kafka L3 engineer. You receive an alert at 3:00 AM: 'Kafka cluster health check FAILED'. Your job is to investigate and resolve all issues within 30 minutes.

Alert Details: Monitoring system reports: Consumer lag on payment-processor-group has grown from 0 to 2,500,000 in the last 15 minutes. Broker-2 is not responding to health checks. 3 partitions of the 'transactions' topic show URP.

Investigation Checklist — Execute in Order

Using the commands you have learned today, execute each step and record your findings:

1. Check cluster health — is there an active controller?

```
kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Finding: _____

2. Check for under-replicated partitions

```
kafka-topics.sh --bootstrap-server localhost:9092 --describe --under-replicated-partitions
```

Finding: _____

3. Check consumer lag for payment-processor-group

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group payment-processor-group
```

Finding: _____

4. Check broker-2 availability

```
kafka-broker-api-versions.sh --bootstrap-server broker-2:9092 2>&1
```

Finding: _____

5. Analyze broker logs for root cause

```
grep -i 'ERROR\|dead\|isr\|IOException' /opt/kafka/logs/server.log | tail -30
```

Finding: _____

Incident Report — Complete After Investigation

Fill in this incident report based on your findings:

Field	Your Answer
Root cause (primary)	
Root cause (secondary)	
Severity level (P1/P2/P3/P4)	
Data loss occurred? (Y/N)	
Affected topics/partitions	
Consumer groups impacted	
Immediate remediation step 1	
Immediate remediation step 2	
Long-term prevention measure	
Estimated recovery time	

Post-Incident Analysis Questions

10. Which metric should have alerted BEFORE the consumer lag reached 2.5M messages? What threshold would you set?

Answer: _____

11. If broker-2 had disk full as root cause, what log message would you expect to see first?

Answer: _____

12. What Burrow status would the payment-processor-group have shown before ERROR? At what point would it have transitioned?

Answer: _____

Lab 7 Completion Checklist

- ☐ Executed all 5 investigation steps in order
- ☐ Recorded findings for each investigation step
- ☐ Completed the full incident report table
- ☐ Answered all 3 post-incident analysis questions
- ☐ Discussed your findings with the instructor/group

Day 3 Assessment Quiz

Answer all questions. Passing score: 75% (15/20 correct). Time: 20 minutes.

Section A: Multiple Choice (1 point each)

13. What should the value of ActiveControllerCount be across an entire Kafka cluster?

- a) 0
- b) 1
- c) Equal to the number of brokers
- d) Equal to the number of topics

Your answer: ____

14. Consumer Lag is calculated as:

- a) Log Start Offset - Current Offset
- b) Log End Offset - Log Start Offset
- c) Log End Offset - Current Offset
- d) Current Offset - Log End Offset

Your answer: ____

15. Which log file would you check for partition leadership election events?

- a) server.log
- b) kafka-request.log
- c) controller.log
- d) log-cleaner.log

Your answer: ____

16. An unclean leader election means:

- a) The election took longer than expected
- b) A non-ISR replica was elected — potential data loss
- c) The controller is not responding
- d) All replicas are out of sync

Your answer: ____

17. What configuration controls how long a follower can lag before being removed from ISR?

- a) min.insync.replicas
- b) replica.fetch.max.bytes
- c) replica.lag.time.max.ms
- d) unclean.leader.election.enable

Your answer: ____

Section B: Short Answer (2 points each)

18. Name three root causes of Under-Replicated Partitions and the first diagnostic step for each.

Answer:
_____
_____
_____
_____
_____

19. Explain the difference between consumer lag that is "stable at 50,000 messages" vs. "growing by 5,000 messages/second". Which requires immediate action and why?

Answer:
_____
_____
_____
_____
_____

20. What does the Burrow status 'STALL' mean and how is it different from 'ERROR'?

Answer:
_____
_____
_____

21. You see the error 'java.io.IOException: No space left on device' in the Kafka broker log. Describe your complete response in 3 steps.

Answer:
_____
_____
_____
_____
_____

22. What is the formula for calculating the maximum sustainable message rate for a consumer, and what are the two ways to increase it?

Answer:
_____
_____
_____

Quiz Answer Key

Question	Answer	Reference
Q1	b) 1	Module 1.3 - Controller Metrics
Q2	c) Log End Offset - Current Offset	Module 1.2 - Consumer Lag
Q3	c) controller.log	Module 4.1 - Log Files
Q4	b) Non-ISR replica elected — potential data loss	Module 1.3 - Unclean Elections
Q5	c) replica.lag.time.max.ms	Module 3.2 - Troubleshooting URPs
Q6-Q10	See instructor answer sheet	Modules 1-5